Homework 5: Ontology-based Semantic Grounding

Chun-yien Chang <ccy@hptp.org>

Spring 2026

Introduction

This homework asks each group to construct a small but semantically explicit ontology for a robot agent with grasping capability. The ontology shall ground perceived objects in the course project environment by connecting object types, task roles, manipulation affordances, and inferable graspability. The expected outcome is a compact and testable semantic layer that allows a robot-oriented knowledge base to answer the question: *which objects in the environment are graspable, and why?*

1 Context and Motivation

The AI Capstone course project exposes students to a Physical AI workflow: real-world data collection, reconstruction in simulation, robot motion generation and data creation, policy training, and inference/evaluation. The predefined entry-level tasks are cup stacking, cutlery arrangement, and toy block collection. These tasks introduce object categories such as cups, knife, fork, plate, toy blocks, and basket. This homework adds a semantic layer to the pipeline by requiring students to describe these objects not only as visual labels or simulation entities, but as typed, queryable, and inferable entities in an ontology.

The technical motivation is that a learned policy may operate over observations, object poses, or action trajectories, but a robot agent also needs explicit semantic knowledge when it must explain, validate, reuse, or generalize its behavior. A semantic grounding layer can state that a blue cup is a physical object, a target object in a stacking task, an object with a grasping affordance, and therefore an inferred graspable object. This makes the distinction between *recognition* and *grounding* explicit.

2 Learning Objectives

After completing this homework, students should be able to:

1. Model basic task objects using RDF, RDFS, and OWL constructs.
2. Distinguish object type, task role, affordance, and object instance.
3. Use OWL class axioms or restrictions to support inference.
4. Query inferred graspable objects with SPARQL.
5. Organize ontology, query, code, and report files in a reproducible GitHub repository.
6. Explain how a semantic layer can complement robot learning pipelines.

3 Conceptual Foundation

RDF, Turtle, OWL, and SPARQL

RDF represents information as subject–predicate–object triples. Turtle is a compact textual syntax for writing RDF graphs, and is therefore suitable for small course ontologies and readable examples [1]. OWL 2 provides vocabulary for defining classes, properties, individuals, restrictions, and formally meaningful

ontology axioms [2], [3]. SPARQL is the standard query language for RDF graphs, including graphs stored natively as RDF or exposed through middleware [4].

Robotics Ontology References

This homework uses a simplified teaching ontology rather than requiring students to import a full robotics ontology. Nevertheless, the design is aligned with major robotics knowledge-representation traditions:

- **IEEE CORA / IEEE 1872-2015**: a core ontology for robotics and automation, intended to provide fundamental concepts from which more detailed robotics ontologies can be constructed [5].
- **IEEE AuR / IEEE 1872.2-2021**: an autonomous robotics ontology that extends CORA toward domain knowledge for autonomous systems [6].
- **SOMA**: the Socio-physical Model of Activities, used for modeling everyday manipulation activities and embodied agents [7].
- **KnowRob**: a robot knowledge processing system designed to support autonomous robots with knowledge needed for everyday manipulation tasks [8], [9].

The course ontology should therefore be understood as a deliberately reduced educational profile: CORA/AuR-inspired at the upper level, SOMA/KnowRob-inspired at the manipulation and affordance level, and course-specific at the task-object level. To keep the course ontology lightweight while still making its terminology traceable, selected course terms may be aligned with existing robotics ontology vocabularies using SKOS. These mappings are used for documentation and semantic cross-reference only; the reasoning behavior required in this homework remains defined by the local OWL axioms. The alignment principles, namespace declarations, and formal expressions are given in the Appendix.

4 Homework Problem Statement

Assume that a robot agent with a gripper observes several objects in a task environment. The robot receives object labels, colors, and possibly pose-frame identifiers from perception or simulation. Your task is to build an ontology that allows the robot to ground these observed objects semantically and infer which objects are graspable.

Your ontology must include at least the object types involved in the three predefined course tasks:

1. **Cup stacking**: blue cup and pink cup.
2. **Cutlery arrangement**: knife, fork, and plate.
3. **Toy block collection**: toy blocks and basket.

Groups working on a self-defined advanced task may add additional object types and affordances, but they must still include the baseline object vocabulary above.

5 Required Ontology Resources

Your ontology must use the following resources at minimum:

- `owl:Class`
- `owl:ObjectProperty`
- `owl:DatatypeProperty`
- `rdfs:subClassOf`
- `rdfs:label`
- `rdfs:comment` or `skos:definition`
- at least one `owl:Restriction` or `owl:equivalentClass` pattern that supports inference
- object instances representing observed or simulated objects

- at least one reasoning result showing inferred `cap:GraspableObject` instances

Each major ontology entity, including classes, object properties, data properties, and task-relevant individuals, must include a human-readable label and a short explanatory annotation. The recommended annotation pattern is `rdfs:label` for a readable name and either `rdfs:comment` or `skos:definition` for a concise explanation of the entity's meaning in the course project context. Groups may also use `skos:scopeNote` when they need to clarify modeling scope, limitations, or intended usage.

These annotations do not replace formal axioms. They make the ontology interpretable to human readers and help explain the modeling commitments behind each term, while OWL/RDFS axioms remain responsible for classification, restriction, and reasoning behavior.

Each major ontology entity, including classes, object properties, data properties, and task-relevant individuals, must include a human-readable label and a short explanatory annotation. The recommended annotation pattern is `rdfs:label` for a readable name and either `rdfs:comment` or `skos:definition` for a concise explanation of the entity's meaning in the course project context. Groups may also use `skos:scopeNote` when they need to clarify modeling scope, limitations, or intended usage.

These annotations do not replace formal axioms. They make the ontology interpretable to human readers and help explain the modeling commitments behind each term, while OWL/RDFS axioms remain responsible for classification, restriction, and reasoning behavior. Examples of the required annotation style are provided in Listings 1, 2, 3, and 4.

6 Recommended Modeling Distinctions

A common modeling error is to treat every task-related object as simply "graspable." This homework requires a more precise distinction:

Layer	Meaning
Object type	Shared course class, e.g., `cap:Cup`, `cap:Knife`, `cap:Fork`, `cap:Plate`, `cap:ToyBlock`, `cap:Basket`.
Task role	Shared course role class, e.g., `cap:TargetObject`, `cap:ReferenceObject`, `cap:ContainerTarget`, `cap:CollectableObject`.
Affordance	Shared course affordance class, e.g., `cap:GraspingAffordance`, `cap:SupportAffordance`, `cap:ContainmentAffordance`, `cap:StackabilityAffordance`.
Instance	Group-specific individual, e.g., `g05:blueCup01`*, `g05:knife01`*, `g05:block01`*.
Inferred class	Reasoner-derived class membership, e.g., `g05:blueCup01`* `rdf:type` `cap:GraspableObject`.

*The prefix `g05:` is used only as an example group namespace. Each group must replace it with its own group-specific prefix and ontology namespace. See Section 8 for the namespace policy.

The distinction is important because a plate may be a placement reference in the cutlery arrangement task, while a knife and fork are direct manipulation targets. A basket may be the container target in block collection, while toy blocks are the collectable grasp targets. Whether a plate or basket should also be graspable depends on how the group models the robot's task and gripper capability.

7 Base Ontology Profile

Students may use the following base ontology profile as a starting point. The prefix `cap:` stands for the course-specific AI Capstone ontology namespace.

7.1 Core Classes

Listing 1 shows the opening excerpt of the complete course ontology. It includes the namespace declarations, ontology-level metadata, and the first part of the core class vocabulary. The metadata block identifies the ontology URI, preferred namespace prefix, title, description, creator, creation date, and license. The class definitions that follow illustrate the expected annotation style and the basic class hierarchy used throughout the homework. The complete ontology is provided as a single Turtle file: course-affordance.ttl.

Listing 1: Opening excerpt of the course ontology, including namespace declarations, ontology metadata, and core class definitions.

```
@prefix cap:  <https://hcis.io/ontology/aicapstone/2026/> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix vann:    <http://purl.org/vocab/vann/> .
@prefix foaf:    <http://xmlns.com/foaf/0.1/> .

# Ontology Metadata

<https://hcis.io/ontology/aicapstone/2026>
    a owl:Ontology ;
    rdfs:label "AI Capstone 2026 Course Ontology"@en ;
    rdfs:comment
        "A reduced educational ontology profile for semantic and affordance grounding
    of graspable objects in the AI Capstone 2026 course project."@en ;
    dcterms:title "AI Capstone 2026 Course Ontology"@en ;
    dcterms:description
        "This ontology defines a compact course vocabulary for representing physical
    task objects, task roles, manipulation affordances, grounding identifiers, and
    inferable graspability in the AI Capstone 2026 Physical AI project."@en ;
    dcterms:creator cap:ccyontologist ;
    dcterms:created "2026-05-22"^^xsd:date ;
    dcterms:license <https://creativecommons.org/licenses/by/4.0/> ;
    vann:preferredNamespacePrefix "cap" ;
    vann:preferredNamespaceUri "https://hcis.io/ontology/aicapstone/2026/" .

cap:ccyontologist
    a foaf:Person ;
    foaf:name "Chun-yien Chang"@en ;
    foaf:mbox <mailto:ccy@hptp.org> .

# Core Classes

cap:PhysicalObject
    a owl:Class ;
    rdfs:label "physical object"@en ;
    rdfs:comment
        "A material object in the task environment that may be perceived, manipulated,
     used as a reference object, or assigned a task role."@en .

# ... omitted for space; see the complete ontology file.
```

7.2 Properties

Listing 2 shows an excerpt of the property definitions from the complete course ontology. The excerpt illustrates the expected annotation style, domain and range declarations, and the difference between ontology-level semantic relations, such as `cap:hasAffordance` and `cap:hasTaskRole`, and pipeline-grounding identifiers, such as `cap:hasObjectLabel` and `cap:hasPoseFrame`.

Listing 2: Excerpt of the object and data properties.

```
# Properties

cap:hasAffordance
    a owl:ObjectProperty ;
    rdfs:domain cap:PhysicalObject ;
    rdfs:range cap:Affordance ;
    rdfs:label "has affordance"@en ;
    rdfs:comment
        "Relates a physical object to an affordance associated with that object in the
     task context."@en .

cap:hasTaskRole
    a owl:ObjectProperty ;
    rdfs:domain cap:PhysicalObject ;
    rdfs:range cap:TaskRole ;
    rdfs:label "has task role"@en ;
    rdfs:comment
        "Relates a physical object to the role it plays in a task."@en .

cap:hasTargetObject
    a owl:ObjectProperty ;
    rdfs:domain cap:Task ;
    rdfs:range cap:PhysicalObject ;
    rdfs:label "has target object"@en ;
    rdfs:comment
        "Relates a task to a physical object that is directly acted upon in that task.
    "@en .

cap:canBeManipulatedBy
    a owl:ObjectProperty ;
    rdfs:domain cap:PhysicalObject ;
    rdfs:range cap:EndEffector ;
    rdfs:label "can be manipulated by"@en ;
    rdfs:comment
        "Relates a physical object to an end effector capable of manipulating it."@en
    .

cap:hasObjectLabel
    a owl:DatatypeProperty ;
    rdfs:domain cap:PhysicalObject ;
    rdfs:range xsd:string ;
    rdfs:label "has object label"@en ;
    rdfs:comment
        "Records the object label used by perception, simulation, or task
    documentation."@en .

# ... omitted for space; see the complete ontology file.
```

8 Ontology URI and Namespace Policy

The course ontology introduced above uses the prefix `cap:` for the shared vocabulary provided in this homework:

```
@prefix cap: <https://hcis.io/ontology/aicapstone/2026/> .
```

This namespace is reserved for course-level terms such as `cap:PhysicalObject`, `cap:GraspableObject`, `cap:hasAffordance`, and `cap:hasPoseFrame`. Groups should reuse these terms when referring to the shared course vocabulary, but they should not place their own task-specific classes, instances, or extensions directly under the `cap:` namespace.

Each group must define its own ontology URI and namespace for its submitted ontology. Groups must refer back to the ontology metadata example in Listing 1 and provide the same kinds of metadata for their own ontology: title, description, creator information, creation date, license, preferred namespace prefix, and preferred namespace URI. For example, a group may define a namespace such as:

```
@prefix g05: <https://hcis.io/ontology/aicapstone/2026/group05/> .
```

In this pattern, `cap:` denotes the shared course ontology, while `g05:` denotes the group's own modeling space. Course terms may be reused directly, but group-specific object classes, task variants, individuals, or additional affordances should be declared under the group namespace.

9 Course Object Layer

The following object layer covers the three entry-level tasks. Listing 3 shows an excerpt of the course-specific object classes from the complete course ontology. Students may extend this layer for advanced tasks by adding additional object classes, affordances, and task roles.

Listing 3: Excerpt of the course-specific object classes.

```
# Course Objects

cap:Cup
    a owl:Class ;
    rdfs:subClassOf cap:PhysicalObject ;
    rdfs:label "cup"@en ;
    rdfs:comment
        "A container-like object used in the cup-stacking task."@en ;
    rdfs:subClassOf [
        a owl:Restriction ;
        owl:onProperty cap:hasAffordance ;
        owl:someValuesFrom cap:GraspingAffordance
    ] ;
    rdfs:subClassOf [
        a owl:Restriction ;
        owl:onProperty cap:hasAffordance ;
        owl:someValuesFrom cap:StackabilityAffordance
    ] .

cap:Knife
    a owl:Class ;
    rdfs:subClassOf cap:PhysicalObject ;
    rdfs:label "knife"@en ;
    rdfs:comment
        "A cutlery object to be placed in the cutlery-arrangement task."@en ;
    rdfs:subClassOf [
```

```
        a owl:Restriction ;
        owl:onProperty cap:hasAffordance ;
        owl:someValuesFrom cap:GraspingAffordance
    ] .

# ... omitted for space; see the complete ontology file.
```

10 Environment Instances

Students must create object instances representing the objects in the task environment. The submitted ontology must include instances for the baseline objects involved in all three predefined tasks, regardless of which entry-level task the group selects: the blue cup and pink cup for cup stacking; the knife, fork, and plate for cutlery arrangement; and the toy blocks and basket for toy block collection. Groups working on a self-defined advanced task must add additional instances for their own task-specific objects.

These individuals should be declared under the group-specific namespace, while their semantic types and properties may reuse the shared `cap:` vocabulary. In the example below, `g05:` is only a sample group prefix. Each group must replace it with its own actual group namespace and use identifiers consistent with its group number or repository design.

Listing 4: Example group-specific object instance.

```
@prefix cap: <https://hcis.io/ontology/aicapstone/2026/> .
@prefix g05: <https://hcis.io/ontology/aicapstone/2026/group05/> .

# In this example, g05: is a sample prefix for Group 05.
# Each group should replace it with its own group-specific namespace.

g05:blueCup01
    a cap:Cup ;
    rdfs:label "blue cup 01"@en ;
    rdfs:comment
        "The blue cup instance defined by Group 05 for the baseline cup-stacking task.
    "@en ;
    cap:hasColor "blue" ;
    cap:hasObjectLabel "blue_cup" ;
    cap:hasTaskRole cap:TargetObject ;
    cap:hasPoseFrame "world/object_blue_cup" .

# Define the remaining baseline task objects using the same pattern:
# pink cup, knife, fork, plate, toy blocks, and basket.
# Add advanced-task objects if the group defines an additional task.
```

11 Reasoning Pattern

After defining the shared vocabulary, course-specific object classes, and environment instances, the next step is to specify how graspability can be inferred. The central reasoning target is `cap:GraspableObject`. Conceptually, the class can be defined in Description Logic as follows:

$$\texttt{cap:GraspableObject} \equiv \texttt{cap:PhysicalObject} \sqcap \exists\, \texttt{cap:hasAffordance}.\texttt{cap:GraspingAffordance}.$$

This means that a graspable object is defined as a physical object that has at least one grasping affordance. In OWL/Turtle, the same modeling pattern can be serialized as an `owl:equivalentClass` axiom with an `owl:intersectionOf` expression and an existential restriction:

Listing 5: OWL/Turtle serialization of the graspable-object reasoning pattern.

```
cap:GraspableObject
    a owl:Class ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            cap:PhysicalObject
            [ a owl:Restriction ;
              owl:onProperty cap:hasAffordance ;
              owl:someValuesFrom cap:GraspingAffordance
            ]
        )
    ] .
```

The intended inference is:

> If an individual is a `cap:PhysicalObject` and has at least one `cap:hasAffordance` relation to `cap:GraspingAffordance`, then the individual can be classified as `cap:GraspableObject`.

For this homework, the required reasoning feature is class classification based on OWL class definitions, subclass axioms, and existential restrictions. In particular, the reasoner should be able to use axioms such as `owl:equivalentClass`, `owl:intersectionOf`, and `owl:someValuesFrom` to classify individuals under `cap:GraspableObject`.

Several workflows can support this requirement. Protégé with an OWL 2 DL reasoner such as HermiT or Pellet is suitable for directly checking inferred class membership. A Java workflow may use Apache Jena for RDF parsing and SPARQL querying, but groups must verify whether the selected Jena inference setup supports the OWL pattern used in their ontology. A Python workflow using RDFLib is acceptable only if the group clearly documents the additional reasoning mechanism used to derive `cap:GraspableObject` membership.

For example, if every `cap:Cup` is modeled as having some grasping affordance, then individuals typed as `cap:Cup` should be classified under `cap:GraspableObject` only when the selected reasoning workflow supports the required existential-restriction pattern.

This requirement follows the OWL modeling constructs used in the definition above; students should consult the documentation of their selected tool to confirm support for the relevant OWL reasoning features [3], [10], [11].

12 Query Requirement

Each group must provide at least one SPARQL query that retrieves inferred graspable objects. The query should be executed over an inferred model, not only the raw asserted graph.

Listing 6: SPARQL query for inferred graspable objects.

```
PREFIX cap: <https://example.org/aicapstone/ontology#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a cap:GraspableObject .
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
  OPTIONAL { ?obj cap:hasTaskRole ?role . }
}
ORDER BY ?obj
```

Expected results should include objects such as `cap:blueCup01`, `cap:pinkCup01`, `cap:knife01`, `cap:fork01`, and `cap:block01`, assuming that the group uses the recommended class axioms. The inclusion of `cap:plate01` or `cap:basket01` depends on whether the group explicitly models them as graspable in its task context.

OWL reasoning and SPARQL querying play different roles in this homework. The OWL reasoning pattern defines `cap:GraspableObject` at the ontology level and allows new class memberships to be inferred. SPARQL, by contrast, retrieves data from the asserted or inferred RDF graph. For example, a SPARQL property path can search for affordance individuals typed as `cap:GraspingAffordance` or as one of its subclasses:

```
?affordance rdf:type/rdfs:subClassOf* cap:GraspingAffordance .
```

However, this remains a query-level graph pattern; it does not by itself establish the ontology-level meaning of `cap:GraspableObject`. Therefore, students should define graspability with OWL, classify individuals through a reasoning workflow, and use SPARQL to retrieve the inferred results.

13 Technology Options

Students may use one of the following workflows. The ontology and query files are required regardless of workflow.

13.1 Option A: Protégé Workflow

1. Create or open the Turtle ontology in Protégé.
2. Inspect class hierarchy, object properties, data properties, and individuals.
3. Run a reasoner such as HermiT or Pellet if available in the local installation.
4. Verify that selected individuals are inferred as `cap:GraspableObject`.
5. Export the ontology as Turtle.
6. Include screenshots of the inferred class membership in the report.

13.2 Option B: Apache Jena Workflow

Apache Jena supports RDF data handling, ontology models, inference, SPARQL querying, SHACL, and persistent triple stores such as TDB2 [11], [12]. A minimal command-line workflow is:

Listing 7: Possible Jena CLI workflow.

```
# Validate Turtle syntax by trying to parse the file.
riot --validate ontology/course-affordance.ttl

# Query a local RDF file. Inference support may require code or Fuseki config.
arq --data ontology/course-affordance.ttl --query queries/graspable_objects.rq
```

For reasoning, students may implement a small Java program that loads the ontology into an inference model and runs the SPARQL query over the inferred graph.

13.3 Option C: Python RDFLib Workflow

RDFLib can parse Turtle and execute SPARQL queries, but full OWL reasoning is not its default strength. Groups choosing Python should either:

- use RDFLib for parsing and querying plus a separate reasoner, or
- explicitly document that their inference is implemented through an additional rule layer rather than complete OWL reasoning.

A Python-only solution is acceptable for this homework only if the group clearly shows how `cap:GraspableObject` membership is derived rather than manually asserted.

13.4 Option D: Fuseki Endpoint Workflow

Advanced groups may load the ontology into Apache Jena Fuseki, configure an inference dataset, and query the endpoint through the web UI or HTTP. This is optional. If used, the repository should include configuration notes and query examples.

14 Minimal Java Example for Jena

The following Java skeleton illustrates the intended workflow. Students may adapt it.

Listing 8: Minimal Jena-style reasoning and query workflow.

```
import org.apache.jena.rdf.model.*;
import org.apache.jena.reasoner.*;
import org.apache.jena.reasoner.rulesys.*;
import org.apache.jena.query.*;
import org.apache.jena.util.FileManager;

public class GraspableQuery {
    public static void main(String[] args) {
        String ontologyPath = "ontology/course-affordance.ttl";
        String queryPath = "queries/graspable_objects.rq";

        Model base = FileManager.getInternal().loadModel(ontologyPath);

        Reasoner reasoner = ReasonerRegistry.getOWLReasoner();
        InfModel infModel = ModelFactory.createInfModel(reasoner, base);

        String queryString = FileManager.getInternal().readWholeFileAsUTF8(queryPath);
        Query query = QueryFactory.create(queryString);

        try (QueryExecution qe = QueryExecutionFactory.create(query, infModel)) {
            ResultSet results = qe.execSelect();
            ResultSetFormatter.out(System.out, results, query);
        }
    }
}
```

If this program does not produce the expected results, students should inspect whether the reasoner supports the exact OWL pattern used in the ontology. They may simplify the pattern or implement a Jena rule, provided that the report explicitly explains the inference mechanism.

15 Optional SHACL Validation

SHACL is not mandatory for this homework, but it is recommended for groups that want to distinguish reasoning from validation. OWL is used to infer class membership; SHACL can be used to check whether the submitted graph satisfies required structural constraints.

Possible validation constraints:

- every `cap:PhysicalObject` instance must have at least one `cap:hasObjectLabel`;
- every task target must have at least one `cap:hasTaskRole`;
- every object intended as a manipulation target must have at least one `cap:hasAffordance`;
- every advanced task must define at least one new object class and one new affordance or task role.

16 Deliverables

Each group must prepare a public or course-accessible GitHub repository and submit its link on the corresponding E3 assignment page. The repository must contain the group-authored ontology, imported ontology files, SPARQL queries, reasoning results, documentation, and report described below.

16.1 GitHub Repository Structure

Listing 9: Required repository structure.

```
semantic-affordance-grounding/
|-- README.md
|-- homework5-report.pdf                # or homework5-report.md
|-- ontology/
|   |-- group-ontology.ttl              # the group's submitted ontology
|   |-- inferred-results.ttl            # exported inferred graph
|   `-- imports/
|       |-- course-affordance.ttl       # provided course ontology
|       `-- ...                         # other imported ontologies, if used
|-- queries/
|   |-- graspable_objects.rq
|   `-- task_objects.rq                 # optional but recommended
|-- results/
|   |-- graspable_objects_output.txt
|   `-- screenshots/                    # recommended for Protege/Fuseki workflows
|-- src/                                # Java or Python code, if used
|   `-- ...
`-- LICENSE                             # optional
```

The main ontology submitted by the group should be `ontology/group-ontology.ttl`. The file `ontology/imports/course-affordance.ttl` is the provided course ontology and should be treated as an imported dependency, not as the group's own ontology. Groups may also place other imported standard ontologies under `ontology/imports/` if they use them explicitly. The README must explain which ontology files are authored by the group and which files are imported resources.

Screenshots are optional but recommended when the reasoning or query result is verified through a graphical interface, such as Protégé or Apache Jena Fuseki. For command-line workflows, a saved text or CSV output file is sufficient as long as the result is reproducible.

16.2 README Requirements

The `README.md` must include:

1. project title and group members;
2. selected task or tasks;
3. a short explanation of the ontology design;
4. a table of modeled objects and their affordances;
5. namespace policy, including which namespace is used for group-specific terms;
6. instructions for running the query;
7. expected query output;
8. explanation of what is inferred, not merely asserted;
9. links to the ontology file, query file, source code, and result files.

16.3 Required Submission Components

Repository item	Requirement
`README.md`	Must explain the repository structure, ontology design, namespace usage, imported resources, query execution, reasoning workflow, and expected results.
`homework5-report.pdf` (or `.md`)	The report should explain the repository contents and the ontology design, including the design rationale, namespace policy, reused and newly introduced terms, key axioms and restrictions, reasoning pattern, query results, design choices, limitations, and any additional issues the group wants to discuss.
`ontology/`	Must contain the group's authored ontology, e.g., `ontology/group-ontology.ttl`. This file should define the group namespace, task instances, required annotations, and any group-specific extensions. If an inferred graph is exported, it may be included as `ontology/inferred-results.ttl`. The `ontology/imports/` folder must include the provided `course-affordance.ttl`, since the course ontology is the required shared vocabulary. Other imported ontologies may also be placed there if explicitly used.
`queries/`	Must contain at least `queries/graspable_objects.rq`, which retrieves inferred `cap:GraspableObject` individuals. Additional queries, such as `queries/task_objects.rq`, are optional but recommended.
`results/`	Must contain saved query output, such as `results/graspable_objects_output.txt`. Screenshots may be placed under `results/screenshots/` when reasoning or query results are verified through a graphical interface such as Protégé or Apache Jena Fuseki.
`src/`	May contain additional code, configuration files, or scripts used in the workflow. This folder is not graded as a separate deliverable, but it should be included when available for troubleshooting and reproducibility.

17 Assessment Rubric

Criterion	Weight	Indicators
Ontology completeness	25%	Required OWL/RDFS resources are present; all baseline task objects are covered; object properties and data properties are meaningful; major ontology entities include readable labels and comments or definitions.
Semantic richness	25%	Object type, task role, affordance, and instance are clearly distinguished; definitions are not merely flat labels.
Reasoning and query	30%	At least one non-trivial inference is used; inferred graspable objects are retrieved by SPARQL; result is reproducible.
Repository and documentation	20%	GitHub repository is organized; README is complete; report explains modeling choices and limitations.

18 Common Modeling Pitfalls

1. **Pitfall: manually asserting all results.** Do not simply write `cap:blueCup01 a cap:GraspableObject`. At least some graspability results must be inferred.
2. **Pitfall: confusing task relevance with graspability.** A basket can be relevant to block collection without being the direct object to grasp.
3. **Pitfall: confusing class and instance.** `cap:Cup` is a class; `cap:blueCup01` is an individual.
4. **Pitfall: overusing labels.** `rdfs:label` or `cap:hasObjectLabel` is not a substitute for class membership and affordance modeling.
5. **Pitfall: querying the raw graph only.** If the query is run without inference, inferred class memberships may not appear.

19 Suggested Extension for Advanced Groups

Advanced groups may extend the ontology in one or more of the following directions:

- add gripper-specific constraints such as object width, mass estimate, or deformability;
- model task success criteria semantically;
- connect object instances to simulation object names or pose frames;
- add SHACL validation for required task-object structures;
- compare ontology-derived graspability with policy success or failure cases.

20 Conclusion

This homework treats ontology engineering as a practical semantic infrastructure for Physical AI. The central requirement is to demonstrate a complete semantic grounding loop: perceived object, class definition, affordance representation, reasoning pattern, inferred graspability, and queryable result. This loop makes object knowledge explicit and therefore inspectable, reusable, and discussable within the robot learning pipeline.

Appendix: SKOS Alignment for the Course Ontology

SKOS, the Simple Knowledge Organization System, is a W3C vocabulary for representing concept schemes, labels, definitions, notes, and mapping relations between terms. In this appendix, SKOS is used as a lightweight conceptual modeling layer for relating selected course ontology terms to terminology from existing robotics or foundational ontologies. This use is appropriate when the goal is semantic traceability rather than full OWL-level axiomatization or reasoning alignment.

The mappings below should therefore be read as conceptual alignments and documentation links. They clarify how the educational vocabulary relates to broader ontology-engineering terminology without requiring the course ontology to reproduce the full formal commitments of the external ontology systems.

Formal Interpretation

Let C be the set of course ontology terms, and let E be the set of external ontology terms. A lightweight alignment relation can be represented as:

$$\mathsf{align} \subseteq C \times R_{\mathrm{SKOS}} \times E$$

where

$$R_{\mathrm{SKOS}} = \{\mathsf{exactMatch}, \mathsf{closeMatch}, \mathsf{broadMatch}, \mathsf{narrowMatch}, \mathsf{relatedMatch}\}.$$

For a course term $c \in C$, an external term $e \in E$, and a SKOS mapping relation $r \in R_{\text{SKOS}}$, an alignment assertion has the form:

$$\text{align}(c, r, e).$$

For example:

$$\text{align}(\texttt{cap:RobotAgent}, \texttt{skos:closeMatch}, \texttt{cora:Robot}).$$

This assertion should be read as a semantic cross-reference between the course term and the external term. The following would be an incorrect interpretation of the SKOS mapping:

$$\texttt{cap:RobotAgent} \equiv \texttt{cora:Robot}. \quad \text{(Incorrect)}$$

Similarly,

$$\text{align}(\texttt{cap:GraspingAffordance}, \texttt{skos:relatedMatch}, \texttt{soma:Grasping})$$

does not imply class equivalence. It indicates only that the course-level notion of grasping affordance is conceptually related to an external term used in manipulation-oriented ontology modeling.

Modeling Principle

The course ontology is intentionally smaller than the external ontology systems to which it refers. SKOS mappings should therefore be interpreted as documentation-level alignments rather than as OWL class axioms. In particular, `skos:exactMatch` should be used only when the two mapped terms have demonstrably equivalent meanings and compatible identity conditions. For this homework, most mappings should use `skos:closeMatch`, `skos:broadMatch`, or `skos:relatedMatch`.

These mappings do not determine the OWL reasoning behavior of this homework. They provide traceability to external terminology, while local OWL axioms define the classification behavior required for terms such as `cap:GraspableObject`.

Namespace Declarations

The following namespace declarations identify external vocabularies that may be referenced for SKOS alignment or further ontology extension. Some prefixes are used in the examples below, while others are included as optional reference namespaces for students who wish to extend the alignment.

Listing 10: External ontology namespaces

```
@prefix skos:      <http://www.w3.org/2004/02/skos/core#> .
@prefix cora:      <http://purl.org/ieee1872-owl/cora-bare#> .
@prefix sumocora:  <http://purl.org/ieee1872-owl/sumo-cora#> .
@prefix soma:      <http://www.ease-crc.org/ont/SOMA.owl#> .
@prefix dul:       <http://www.ontologydesignpatterns.org/ont/dul/DUL.owl#> .
@prefix corax:     <http://purl.org/ieee1872-owl/corax#> .
@prefix knowrob:   <http://knowrob.org/kb/knowrob.owl#> .
```

Example SKOS Alignment

Listing 11 shows an excerpt of the SKOS alignment used for the course ontology. The excerpt illustrates how selected course terms may be documented and related to external ontology vocabularies. The complete alignment file is available at: `course-alignment.ttl`.

Listing 11: Excerpt of SKOS alignment for selected course ontology terms.

```
cap:RobotAgent
    skos:prefLabel "robot agent"@en ;
    skos:definition
        "A robot or robot-controlled agent that can perceive, reason about, and
    manipulate objects in the task environment."@en ;
    skos:closeMatch cora:Robot ;
    skos:relatedMatch sumocora:Agent ;
    skos:scopeNote
        "The course term is aligned with CORA at the level of robot-as-agent or robot-
    as-system. It remains locally defined for the purposes of this homework."@en .

cap:ManipulationTask
    skos:prefLabel "manipulation task"@en ;
    skos:definition
        "A robot task in which an agent changes the pose, state, or arrangement of
    physical objects."@en ;
    skos:broadMatch dul:Task ;
    skos:relatedMatch soma:Manipulating ;
    skos:scopeNote
        "The course term covers the project tasks, including cup stacking, cutlery
    arrangement, and toy block collection."@en .

cap:GraspingAffordance
    skos:prefLabel "grasping affordance"@en ;
    skos:definition
        "An affordance by which an object can be grasped, held, lifted, or
    repositioned by a robot gripper."@en ;
    skos:relatedMatch soma:Disposition ;
    skos:relatedMatch soma:Grasping ;
    skos:scopeNote
        "The course term is represented locally as an affordance class. SOMA models
    affordance-like knowledge through dispositions and task-oriented action
    descriptions; therefore relatedMatch is used rather than exactMatch."@en .

# ... additional alignments omitted; see the complete alignment file.
```

References

[1] W3C RDF Working Group, *RDF 1.1 Turtle: Terse RDF Triple Language*, W3C Recommendation, 2014. [Online]. Available: https://www.w3.org/TR/turtle/.

[2] W3C OWL Working Group, *OWL 2 Web Ontology Language Document Overview*, W3C Recommendation, 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/.

[3] W3C OWL Working Group, *OWL 2 Web Ontology Language Primer*, W3C Recommendation, 2012. [Online]. Available: https://www.w3.org/TR/owl2-primer/.

[4] W3C SPARQL Working Group, *SPARQL 1.1 Query Language*, W3C Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/sparql11-query/.

[5] IEEE Standards Association, *IEEE 1872-2015: IEEE Standard Ontologies for Robotics and Automation*, IEEE Standard, 2015. [Online]. Available: https://standards.ieee.org/standard/1872-2015.html.

[6] IEEE Standards Association, *IEEE 1872.2-2021: IEEE Standard for Autonomous Robotics Ontology*, IEEE Standard, 2021.

[7] EASE CRC, *SOMA: Socio-physical Model of Activities Documentation*, Ontology documentation, 2026. [Online]. Available: https://ease-crc.github.io/soma/.

[8] M. Tenorth and M. Beetz, "KnowRob: A Knowledge Processing Infrastructure for Cognition-enabled Robots," *The International Journal of Robotics Research*, vol. 32, no. 5, pp. 566–590, 2013. DOI: 10.1177/0278364913481635.

[9] M. Beetz, D. Beßler, A. Haidu, M. Pomarlan, A. K. Bozcuoglu, and G. Bartels, "KnowRob 2.0: A 2nd Generation Knowledge Processing Framework for Cognition-enabled Robotic Agents," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 512–519. DOI: 10.1109/ICRA.2018.8460964.

[10] The HermiT Development Team, *HermiT OWL Reasoner*, http://www.hermit-reasoner.com/, OWL 2 DL reasoner.

[11] The Apache Software Foundation, *Reasoners and Rule Engines: Jena Inference Support*, https://jena.apache.org/documentation/inference/, Apache Jena documentation.

[12] Apache Jena Project, *Apache Jena Documentation*, Project documentation, 2026. [Online]. Available: https://jena.apache.org/documentation/.